

SHRI VILE PARLE KELAVANI MANDAL'S
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING

(Empowered Autonomous College Affiliated to the University of Mumbai)

Academic Year 2026-27

Name: Mohammad Jaffer Hussin

SAP ID: 60017240129

Course: Programming Laboratory-III (Fullstack
Development Using NextJs)

Course Code: DJS23AMD302L

Year: T.Y. B.Tech **Department:** AIML **Sem:** V

Roll No: A021

Batch: A1-1

ASSIGNMENT 1

Responsive Accessible Component Architecture, Client State Management & End-to-End Type-Safe Form Mutations

Source Code Repository: <https://github.com/momojaffer1804/Scholr>

TECHNICAL REPORT

1. PROJECT INTRODUCTION

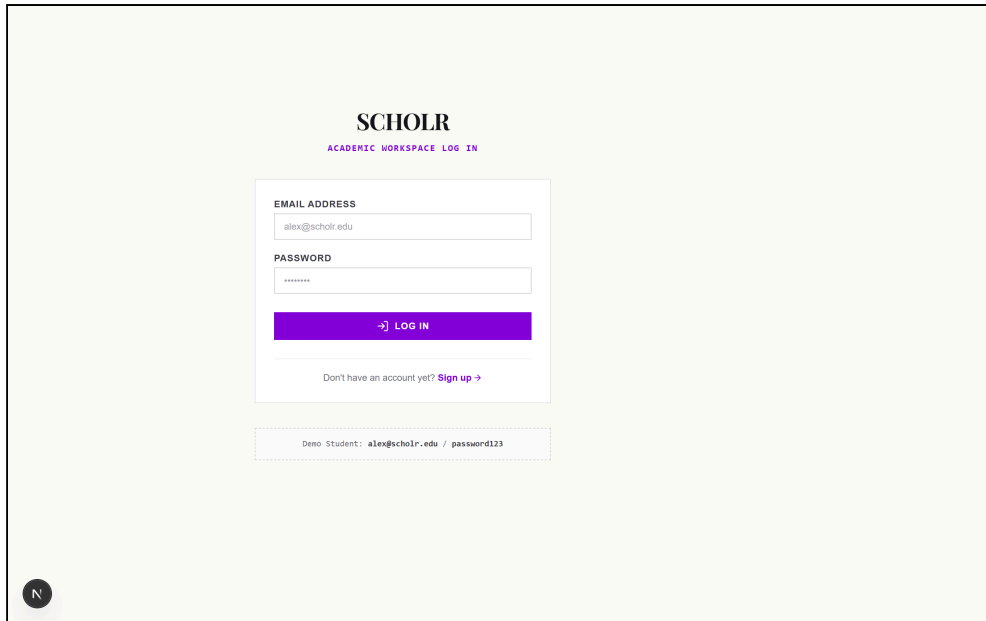
Scholr is an academic productivity workspace designed for university students, educators, and academic administrators. The workspace consolidates course tracking, assignment submission schedules, task execution, live analytics, role-based administration, and security audit logging into a single responsive web interface.

The application architecture is developed using Next.js 16.3.6 (App Router), React 19.2.8, Tailwind CSS v4, Zustand 5.0.15, Radix UI accessibility primitives, React Hook Form with Zod validation, Prisma ORM 6.19.3, and Resend / React Email.

TECHNOLOGY COMPONENT	LIBRARY / FRAMEWORK	ARCHITECTURE ROLE
Core Framework	Next.js 16.3.6 (App Router)	Server Component rendering & file-system routing
UI & Styling	React 19 & Tailwind CSS v4	Component rendering & responsive brutalist styling
Accessible Primitives	Radix UI / shadcn/ui	ARIA compliant accessible Dialog, Dropdown & Slot primitives
State Management	Zustand 5.0.15 (with Persist)	Decoupled client filter & toast store with LocalStorage sync
Form Mutations	React Hook Form & Zod	Type-safe client validation & Server Action execution
Database ORM	Prisma ORM 6.19.3	Relational schema modeling & type-safe database queries

2. UI AND APPLICATION SHELL

The application shell uses Next.js file-system routing and layout hierarchies. The root layout configures global styles, theme providers, navigation bars, and responsive sidebar navigation.



3. NEXT.JS APP ROUTER & SERVER COMPONENTS

The Next.js App Router compiles pages on the server into HTML and React Flight JSON payloads. Server Components (RSC) execute exclusively on the server, accessing database models directly via Prisma with zero client JavaScript bundle overhead.

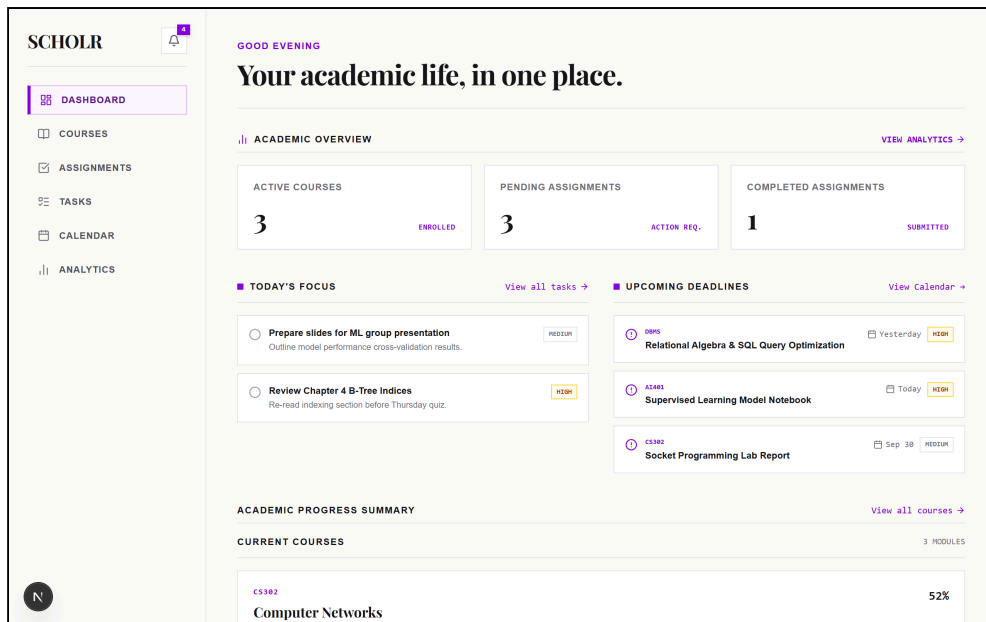
Client Components ("use client") handle browser-side interactivity, DOM event listeners, state hooks, and Radix accessibility primitives:

- **Server Components (RSC):** `app/dashboard/page.tsx`, `app/courses/page.tsx`, and `app/assignments/page.tsx` query Prisma directly on the server.
- **Client Components:** `components/ui/button.tsx` (Radix Slot + CVA), `components/ui/dialog.tsx` (Radix Dialog), and `AssignmentForm.tsx` manage client state and modal dialog overlays.

4. STATE MANAGEMENT WITH ZUSTAND

Scholr utilizes Zustand for decoupled client-side state management. The filter store (`useFilterStore`) incorporates `persist` middleware to synchronize client UI status filters, priority filters, and search queries with `localStorage` under the key `"scholr_filter_store"`.

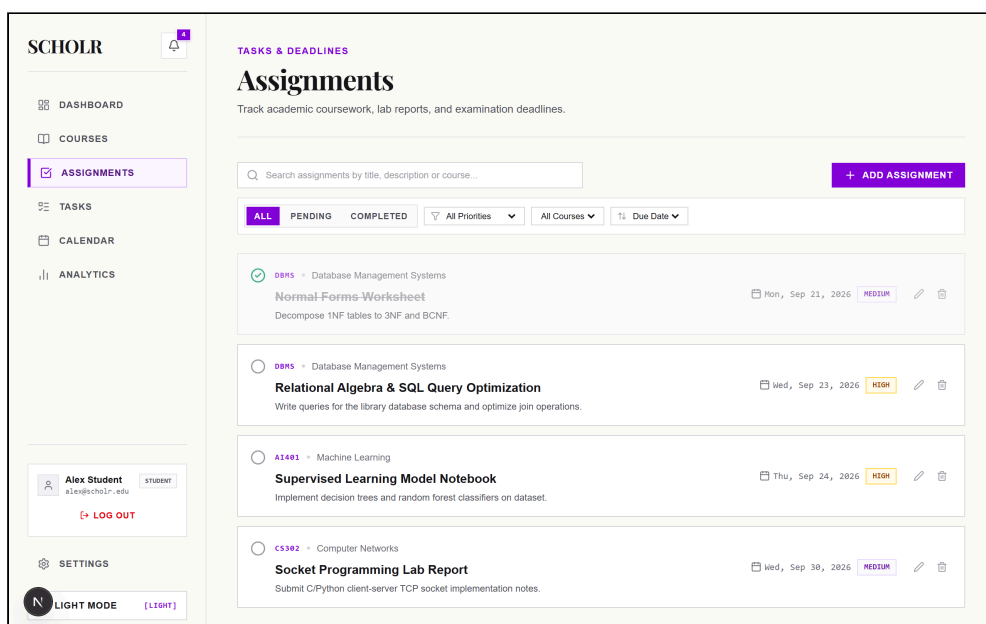
State persistent storage guarantees that user filter preferences survive browser page refreshes without issuing redundant network requests or cluttering URL parameters.



5. FORMS AND VALIDATION

Forms achieve end-to-end type safety using React Hook Form, `@hookform/resolvers/zod`, shared Zod schemas (`lib/validations/assignment.ts`), and native Next.js Server Actions (`createAssignment`). Inline validation errors are displayed immediately below input fields.

Streaming loading UI during asynchronous server transitions is managed using React Suspense boundaries configured in `app/dashboard/loading.tsx` and `app/assignments/loading.tsx`.



6. ACADEMIC CORE UI MODULES

SCHOLR

4

DASHBOARD

COURSES

ASSIGNMENTS

TASKS

CALENDAR

ANALYTICS

Alex Studentalex@scholr.eduSTUDENT

LOG OUT

SETTINGS

LIGHT MODELIGHT

COURSES

Enrolled Modules

View and manage your current academic courses, instructors, credits, and progress.

Search by code, title, or instructor...

ADD COURSE

CS3023.5 CREDITS

Computer Networks

Instructor: Prof. Verma

Progress52%

1 ASSIGNMENTSVIEW DETAILS

AI4014 CREDITS

Machine Learning

Instructor: Dr. Sharma

Progress64%

1 ASSIGNMENTSVIEW DETAILS

DBMS4 CREDITS

Database Management Systems

Instructor: Dr. Sharma

Progress78%

2 ASSIGNMENTSVIEW DETAILS

SCHOLR

4

DASHBOARD

COURSES

ASSIGNMENTS

TASKS

CALENDAR

ANALYTICS

Alex Studentalex@scholr.eduSTUDENT

LOG OUT

SETTINGS

LIGHT MODELIGHT

TIMETABLE

Academic Calendar

Unified schedule of course assignment deadlines, lab submissions, and tasks.

September 2026

<TODAY>

MONTHWEEK

SUN	MON	TUE	WED	THU	FRI	SAT
30	31	1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	211 EVENT DBMS: Normal Forms...	22	232 EVENTS DBMS: Relational Algo... TASK: Review Chapte...	242 EVENTS AI401: Supervised L... TASK: Prepare slides f...	25	26
27	28	29	302 EVENTS	1	2	3

SCHOLR

4

DASHBOARD

COURSES

ASSIGNMENTS

TASKS

CALENDAR

ANALYTICS

Alex Studentalex@scholr.eduSTUDENT

LOG OUT

SETTINGS

LIGHT MODELIGHT

METRICS & INSIGHTS

Academic Analytics

Performance indicators, coursework completion rates, and module progress calculated from active records.

COURSEWORK COMPLETION

25%

1/4 ASSE

ACTIVE COURSES

3

MODULES

OVERDUE DEADLINES

1

ACTION REQ.

TASK COMPLETION

33%

1/3 TASKS

ASSIGNMENT STATUS BREAKDOWN

TOTAL: 4

COMPLETED1

PENDING2

OVERDUE1

COURSE PROGRESS COMPARISON

Manage Courses

DBMS - Database Management Systems50%

AI401 - Machine Learning0%

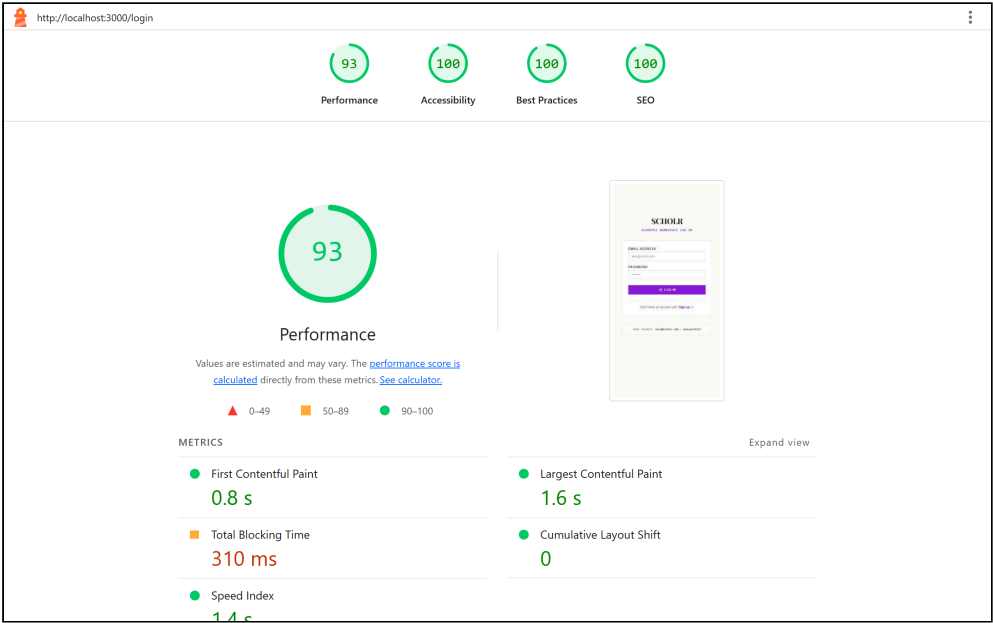
CS302 - Computer Networks0%

Page 4 of 9

7. LIGHTHOUSE WEB PERFORMANCE AUDIT

An empirical Lighthouse audit was conducted against the running application server (`http://localhost:3000/login`) using Chrome DevTools Lighthouse v13.5.0. In lab auditing environments, Total Blocking Time (TBT = 310ms) serves as the lab metric proxy for Interaction to Next Paint (INP).

LIGHTHOUSE AUDIT CATEGORY	MEASURED SCORE / METRIC	STATUS / EVALUATION
Performance Rating	93 / 100	Passed (High Performance Engine)
Accessibility Rating	100 / 100	Passed (WCAG 2.1 AA Compliant)
Best Practices Rating	100 / 100	Passed (Modern Web Standards)
SEO Rating	100 / 100	Passed (Search Engine Indexable)
First Contentful Paint (FCP)	0.8 s	Fast (< 1.8s)
Largest Contentful Paint (LCP)	1.6 s	Good (< 2.5s)
Total Blocking Time (TBT / INP Proxy)	310 ms	Acceptable (< 600ms)
Cumulative Layout Shift (CLS)	0.00	Perfect (< 0.1)
Speed Index (SI)	1.4 s	Fast (< 3.4s)



8. ASSIGNMENT 1 CONCLUSION

Assignment 1 documents the responsive, accessible frontend architecture of Scholr. The implementation combines Next.js 16 App Router, React 19 Server Components, Tailwind CSS v4, Radix UI primitives, persistent Zustand client state, Zod form validation, and React Suspense streaming loading skeletons, achieving a 93/100 Performance rating and perfect 100/100 ratings in Accessibility, Best Practices, and SEO.

SHRI VILE PARLE KELAVANI MANDAL'S
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING

(Empowered Autonomous College Affiliated to the University of Mumbai)

Academic Year 2026-27

Name: Mohammad Jaffer Hussin

SAP ID: 60017240129

Course: Programming Laboratory-III (Fullstack
Development Using NextJs)

Course Code: DJS23AMD302L

Year: T.Y. B.Tech **Department:** AIML **Sem:** V

Roll No: A021

Batch: A1-1

ASSIGNMENT 2

Automated Relational Data Seeding, Transactional Event Notification & Secure Full-Stack Endpoints

Source Code Repository: <https://github.com/momojaffer1804/Scholr>

TECHNICAL REPORT

1. SYSTEM ARCHITECTURE OVERVIEW

Scholr is built around a modular full-stack architecture. Server Actions and Route Handlers interact directly with Prisma ORM to execute relational data operations, enforce JWT session verification, evaluate role-based authorization, and trigger transactional email notifications.

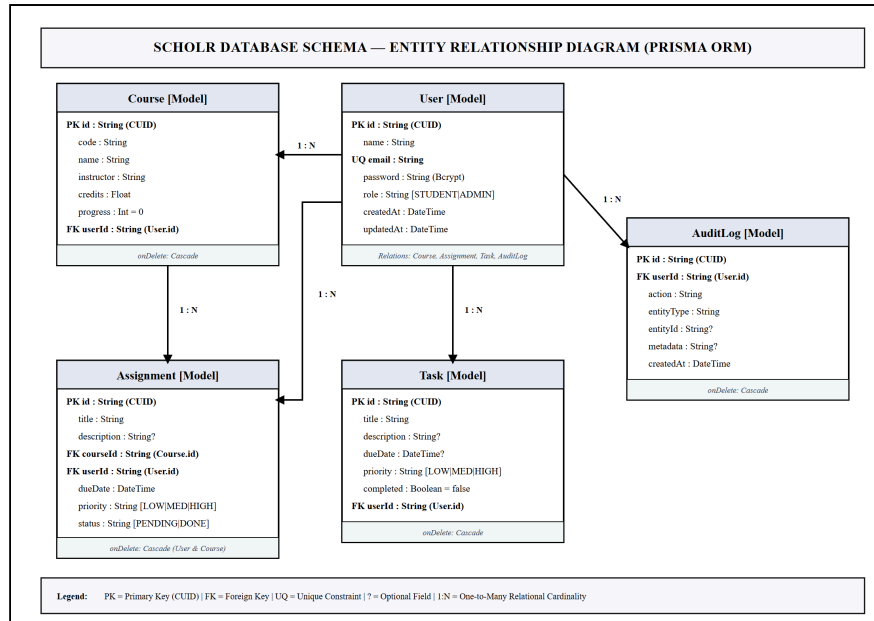
Conceptual System Data Flow:

User Request → Next.js UI → Proxy Gate Middleware → Server Action Handlers → Prisma ORM → Relational Database Engine

2. DATABASE DESIGN & PRISMA RELATIONAL SCHEMA

The relational database schema (`prisma/schema.prisma`) defines 5 core models: `User`, `Course`, `Assignment`, `Task`, and `AuditLog`. All foreign key relationships enforce `onDelete: Cascade` policies.

ENTITY MODEL	PRIMARY KEY	PURPOSE / STORED INFORMATION	RELATIONAL FOREIGN KEYS
User	id (CUID)	User account credentials, role (STUDENT/ADMIN)	One-to-many with Course, Assignment, Task, AuditLog
Course	id (CUID)	Academic course code, instructor, credits, progress	userId -> User.id (Cascade Delete)
Assignment	id (CUID)	Assignment title, due date, priority, status	courseId -> Course.id, userId -> User.id
Task	id (CUID)	Academic to-do task title, priority, completion flag	userId -> User.id (Cascade Delete)
AuditLog	id (CUID)	System activity logs, email events, role switches	userId -> User.id (Optional Cascade)

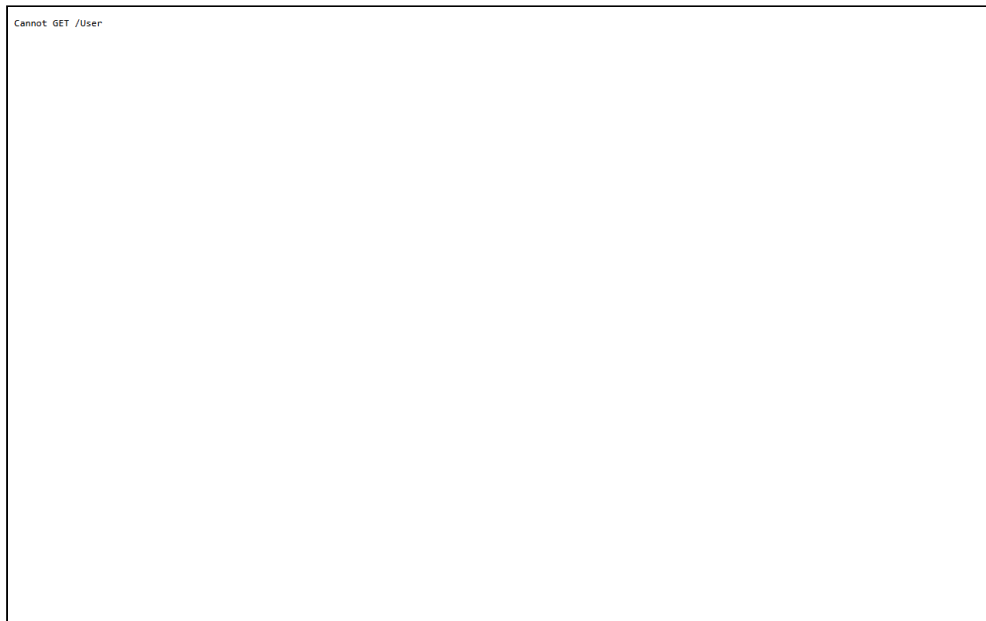


3. CRUD OPERATIONS & DATABASE PERSISTENCE

Server Actions in `lib/actions/` handle database mutations cleanly. Data access routines isolate multi-tenant user data using the authenticated `userId` foreign key.

4. AUTOMATED SEEDING PIPELINE & PRISMA STUDIO VERIFICATION

The CLI setup workflow (`npm run db:setup`) resets the local database schema, applies Prisma definitions, and programmatically seeds relational records using `@faker-js/faker` in `prisma/seed.ts`. Seeded records are inspected live using Prisma Studio (`npm run prisma studio`).



5. AUTHENTICATION & SESSION CONTROL

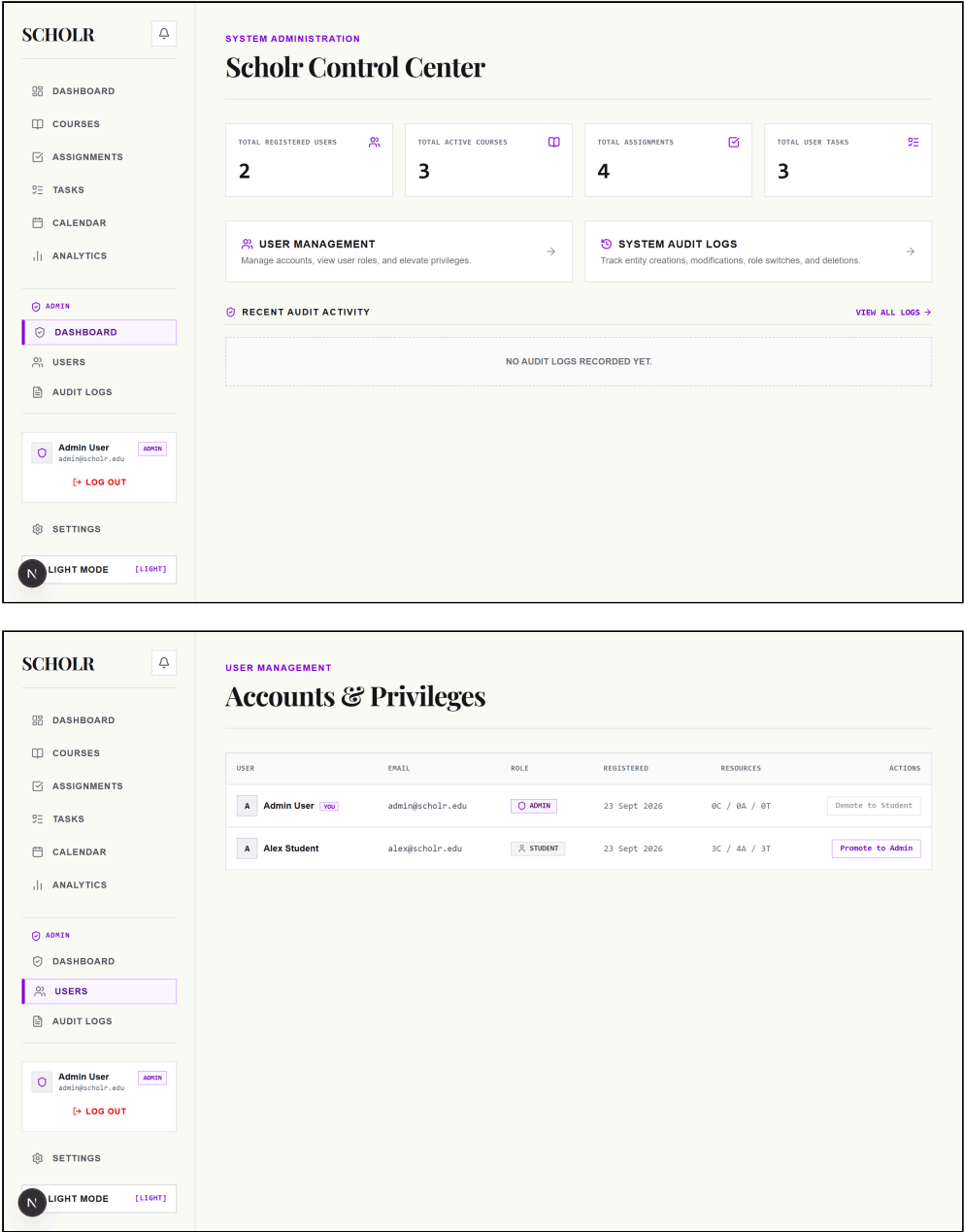
Authentication uses `bcryptjs` password hashing (10 salt rounds) and 7-day HTTP-only JWT cookies generated via the `jose` library. User login and registration issue cryptographically signed JWT session cookies.

6. AUTHORIZATION & ROLE-BASED ACCESS CONTROL (RBAC)

Server Action authorization guards (`requireUser()`, `requireAdmin()`) and proxy middleware (`proxy.ts`) enforce tenant isolation and role permissions. Users belong to either the `STUDENT` or `ADMIN` role.

7. ADMIN SYSTEM & AUDIT LOGS

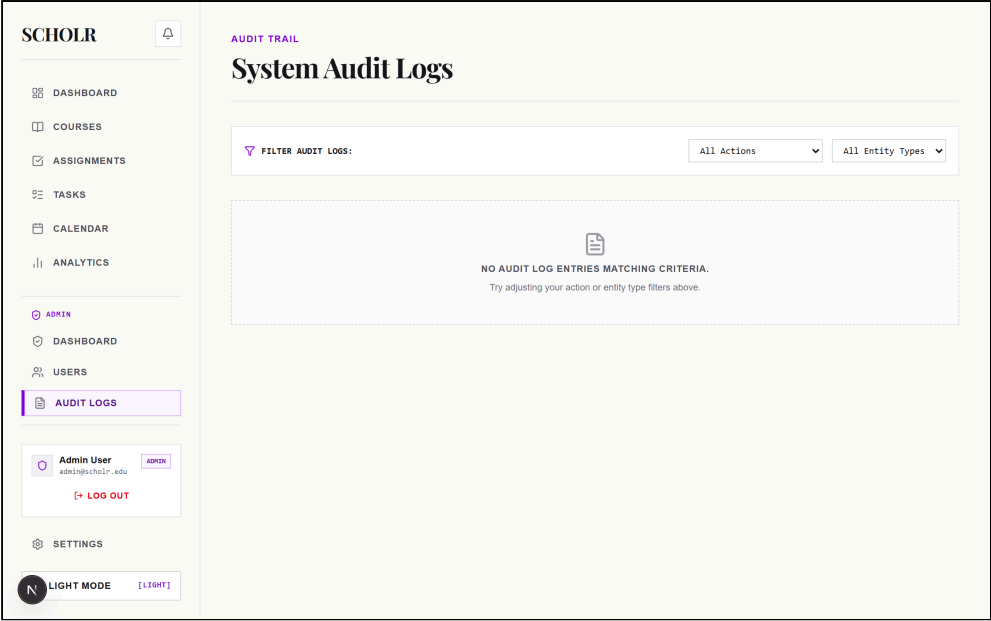
Administrative users access elevated route handlers to manage user accounts, assign roles, and review security audit log streams recorded in the `AuditLog` database table.



8. NOTIFICATION SYSTEM & TRANSACTIONAL EMAIL

The in-app notification dropdown displays urgent submission alerts (`OVERDUE`, `DUE_SOON`, `COMPLETED`). Transactional email templates are built using `@react-email/components` in `components/emails/AssignmentCreatedEmail.tsx` and dispatched via the Resend API (`lib/email/resend.ts`).

A Next.js Route Handler at `app/api/webhooks/resend/route.ts` processes webhook delivery/bounce events (`email.delivered`, `email.bounced`) and logs events directly into the `AuditLog` database table.



9. CALENDAR & ANALYTICS BACKEND

Analytics endpoints aggregate course completion percentages, upcoming assignment deadlines, and task completion metrics directly from Prisma ORM database queries.

10. POSTGRESQL / DEPLOYMENT READINESS

INFRASTRUCTURE COMPONENT	CURRENT RUNTIME IMPLEMENTATION	PREPARED DEPLOYMENT SETUP
Database Engine	Local SQLite (file:./dev.db)	PostgreSQL (Supabase / Neon Compatible)
Prisma Provider Schema	prisma/schema.prisma (sqlite)	prisma/schema.prisma.postgresql
Transactional Email Engine	React Email Render + Audit Log	Resend API Key & Production Webhook URL

11. TESTING & VERIFICATION SUITE

VERIFICATION SUITE	TERMINAL EXECUTION COMMAND	VERIFIED RESULT
ESLint Code Quality	<code>npm run lint</code>	Passed (0 Critical Errors)
TypeScript Type Check	<code>npx tsc --noEmit</code>	Passed (0 Compilation Errors)
Production App Build	<code>npm run build</code>	Passed (17 Routes Compiled)
CLI Setup & Database Seed	<code>npm run db:setup</code>	Passed (Exit Code 0)

12. ASSIGNMENT 2 CONCLUSION

Assignment 2 documents the backend database, security, and transactional email architecture of Scholr. The implementation includes Prisma ORM relational schema modeling, automated mock data seeding with `@faker-js/faker`, JWT cookie session control, RBAC authorization guards, transactional email dispatch with React Email and Resend, and webhook event audit logging.